# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

# 实验九 ContentProvider

## 一、实验目的及任务

1、ContentProvider 的使用
2、Uri 的使用
3、ContentResolover 的使用
4、ContentObserver 的使用

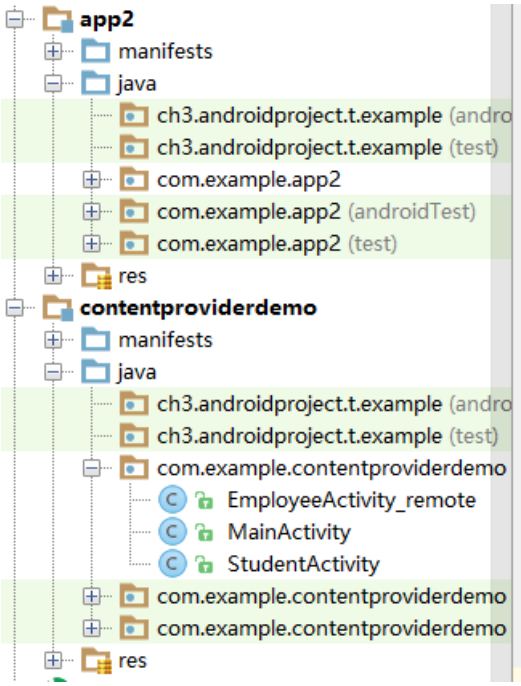
## 二、实验环境

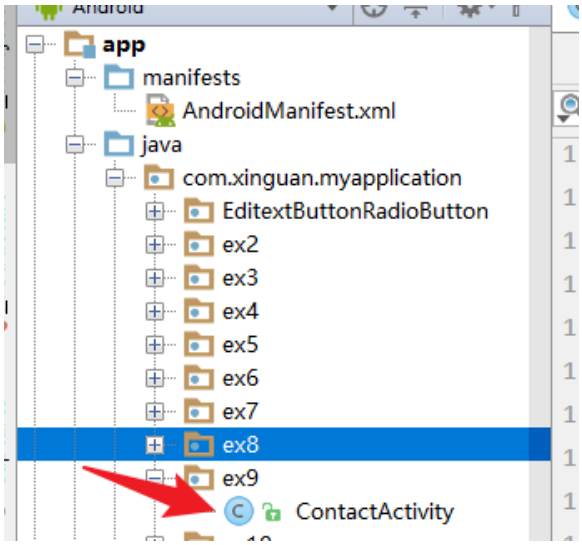
1、 Jdk

2、 Android Studio

## 三、 实验步骤

目录结构：



# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

---

员工信息界面（Module app2）

图 1 员工信息录入界面

图 2 重复添加编号相同的员工失败提示

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 3 成功添加的页面

图 4 清除显示

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 5 选择列表项的效果

图 6 修改成功效果

图 7 删除成功效果

图 8 全部删除

另一个应用程序的界面（**Module contentproviderdemo**）

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 9 初始化效果（“开始”提示）

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 10 远程查询

图 11 点击列表项的效果

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 12 更新后的效果

图 13 模糊查询编号含 1 的员工

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 14 模糊查询员工编号含 2 的员工

图 15 更新的效果

图 16 远程删除的效果

**2、调用系统 URI 查询、添加、删除、修改联系人。**

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 17 当联系人姓名为空时，查询所有联系人

图 18 添加联系人成功效果

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 20 添加联系人后，查询全部联系人

图 21 删除联系人

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

图 22 删除联系人后重新查询

图 23 根据联系人姓名 3 进行模糊查询

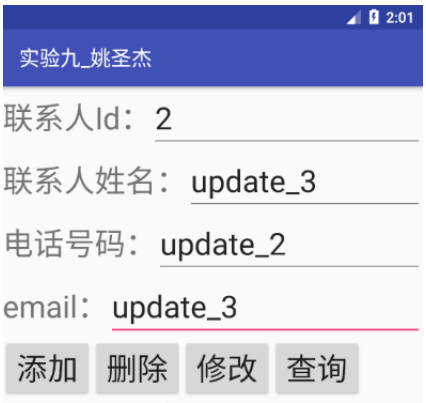


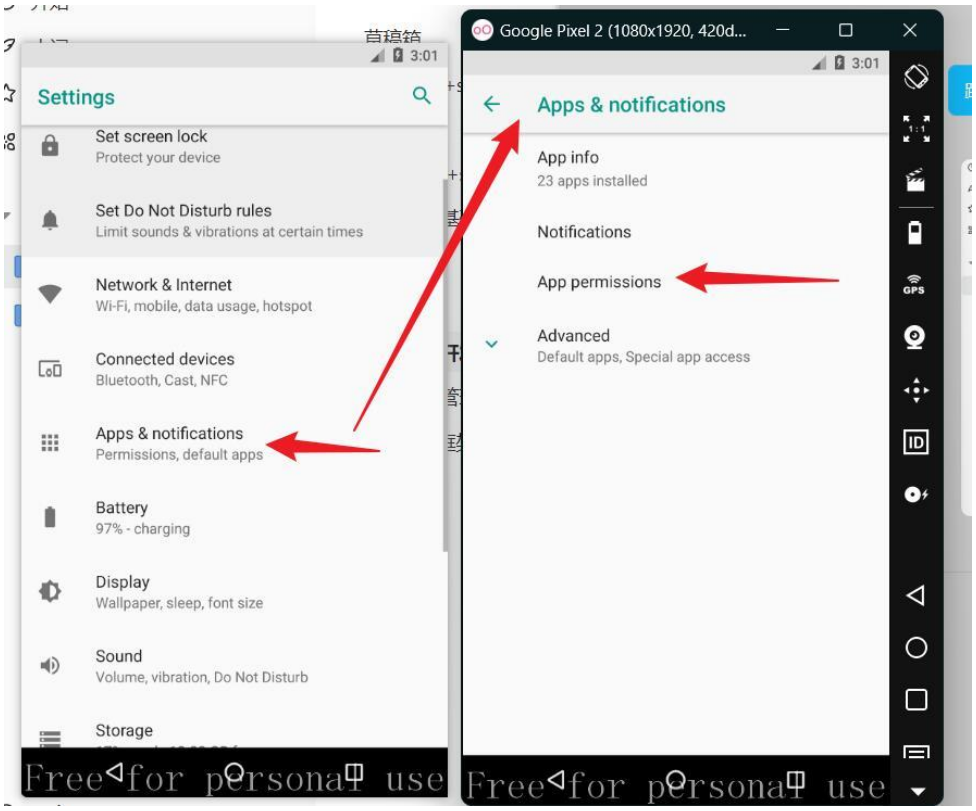

图 24 修改联系人

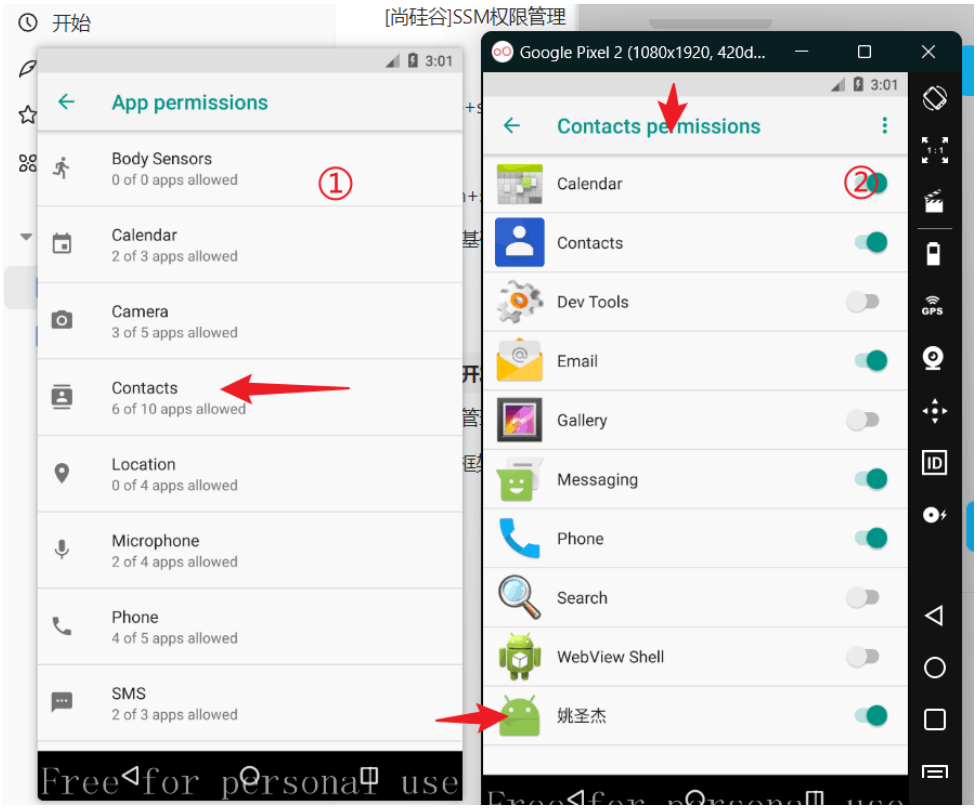
图 25 修改联系人后查询所有

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29



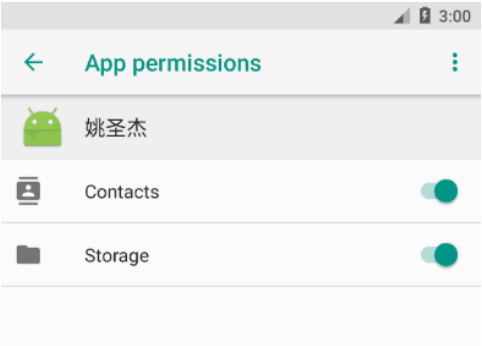



图 26 打开访问“联系人”权限的步骤

## 四、 实验报告

**第一部分：通过自定 contentProvider 远程操作**

**发出者**

App2/AndroidManifest.xml（提供 ContentProvider）

```
<activity android:name=".EmployeeActivity"
    android:label="实验九_姚圣杰">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<provider
    android:name=".StudentContentProvider"
    android:authorities="weili.org.myapplication.studenttest"
    android:exported="true"
    />
<provider
    android:authorities="com.example.employee"
    android:name=".EmployeeContentProvider"
    android:exported="true"
    />
```

**EmployeeDao.java**

```
public class EmployeeDao {

    public static void insert(Context context, String id, String name, String
age, String height){
        EmployeeDatabaseHelper employeeDatabaseHelper=new
EmployeeDatabaseHelper(context);
        SQLiteDatabase db = employeeDatabaseHelper.getWritableDatabase();
        String sql="insert into
employee(employeeid,employeename,employeeage,employeeheight)
values(?,?,?,?)";
        db.execSQL(sql,new Object[]{id,name,age,height});
    }
    public static void delete(Context context,String id){
        EmployeeDatabaseHelper employeeDatabaseHelper=new
EmployeeDatabaseHelper(context);
        SQLiteDatabase db=employeeDatabaseHelper.getWritableDatabase();
        String sql="delete from employee where employeeid=?";
        db.execSQL(sql,new Object[]{id});
    }
    public static void deleteAll(Context context){
        EmployeeDatabaseHelper employeeDatabaseHelper=new
EmployeeDatabaseHelper(context);
        SQLiteDatabase db=employeeDatabaseHelper.getWritableDatabase();
        db.delete("employee",null,null);
    }
    public static void update(Context context,String id, String name, String
age, String height){
        EmployeeDatabaseHelper employeeDatabaseHelper=new
EmployeeDatabaseHelper(context);
        SQLiteDatabase db=employeeDatabaseHelper.getWritableDatabase();
        String sql="update employee set
employeename=?,employeeage=?,employeeheight=?  where employeeid=?";
        db.execSQL(sql,new Object[]{name,age,height,id});
    }
    public static List<Map<String,String>> listAll(Context context){
        EmployeeDatabaseHelper employeeDatabaseHelper=new
EmployeeDatabaseHelper(context);
        List<Map<String,String>> list=new ArrayList<>();
        String sql="select employeeid,employeename,employeeage,employeeheight
from employee";
```

```
        SQLiteDatabase db=employeeDatabaseHelper.getReadableDatabase();
        Cursor cursor=db.rawQuery(sql,null);
        while (cursor.moveToNext()){
            Map<String,String> map =new HashMap<>();
            map.put("employeeid",cursor.getString(0));
            map.put("employeename",cursor.getString(1));
            map.put("employeeage",cursor.getString(2));
            map.put("employeeheight",cursor.getString(3));
            list.add(map);
        }
        return list;
    }
}
```

**EmployeeDatabaseHelper.java**

```
/**
 * Created by 27682 on 2022/11/27.
 * 数据库辅助类，创建员工表
 */

public class EmployeeDatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASENAME="employeeDB.db";//使用Sqlite 数据库，创建TeacherDB，包含teacher 表
    public static final int DATABASEVERSION=1;
    public static final String TABLENAME="employee";
    public EmployeeDatabaseHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version) {
        super(context, name, factory, version);
    }

    public EmployeeDatabaseHelper(Context context) {
        super(context,DATABASENAME,null,DATABASEVERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String sqlCreateTableEmployee="create table employee(employeeid text primary key,employeename text,employeeage text,employeeheight text)";
        db.execSQL(sqlCreateTableEmployee);
    }
```

```
    @Override
    public void onUpgrade(SQLiteDatabase sqLiteDatabase, int i, int i1) {
        String sqlCreateTableTeacher="create table teacher(teacherid text
primary key,teachername text,sex text,age integer)";
        if(i==1&&i1==2)//当是版本号是从1升级到2的时候执行下面的创建teacher表的
操作
            sqLiteDatabase.execSQL(sqlCreateTableTeacher);
    }
}
```

**【将员工的增删改查暴露出去】EmployeeContentProvider.java**

```
public class EmployeeContentProvider extends ContentProvider {
    private UriMatcher matcher=new UriMatcher(UriMatcher.NO_MATCH);
    private EmployeeDatabaseHelper employeeDatabaseHelper;
    final String AUTHORITY="com.example.employee";
    public EmployeeContentProvider(){
        System.out.println("EmployeeContentProvider 构造方法()被调用");
    }
    @Override
    public boolean onCreate() {
        employeeDatabaseHelper=new
EmployeeDatabaseHelper(this.getContext());
        matcher.addURI(AUTHORITY,"updateemployee",1);//修改员工信息
        matcher.addURI(AUTHORITY,"deletemployee/#",1);//删除单个员工，需要根据
employeeid 来判断
        matcher.addURI(AUTHORITY,"deleteallemployee",2);//删除所有员工
        matcher.addURI(AUTHORITY,"employeeall",1);//查询所有员工记录
        matcher.addURI(AUTHORITY,"employees/*",2);//模糊查询多个员工
        matcher.addURI(AUTHORITY, "employee/#", 3);//查询单个员工，需要根据
employeeid 来判断,#是数字通配符，将来可以匹配student/001,student/002 等
        matcher.addURI(AUTHORITY, "insertemployee", 1);//插入员工
        matcher.addURI(AUTHORITY,"updateemployee",1);//更新员工
        return true;
    }

    @Nullable
    @Override
    public Cursor query(@NonNull Uri uri, @Nullable String[] strings, @Nullable
String s, @Nullable String[] strings1, @Nullable String s1) {
        Cursor cursor=null;
        SQLiteDatabase db=employeeDatabaseHelper.getReadableDatabase();
```

```
        switch (matcher.match(uri)){
            case 1://查询所有员工
                cursor = db.query("employee", strings, s, strings1, "", "", s1);
                break;
            case 2://根据员工编号模糊查询
                cursor=db.query("employee", strings, s, strings1, "", "", s1);
                break;
            case 3:
                cursor=db.query("employee", strings, s, strings1, "", "", "");
                break;
            default:
                System.out.println("调用 query 方法时，没有收到合适的 uri");
        }
        return cursor;
    }

    @Nullable
    @Override
    public String getType(@NonNull Uri uri) {
        return null;
    }

    @Nullable
    @Override
    public Uri insert(@NonNull Uri uri, @Nullable ContentValues contentValues)
{
        SQLiteDatabase db=employeeDatabaseHelper.getReadableDatabase();
        switch (matcher.match(uri)){
            case 1:
                db.insert("employee",null,contentValues);
                db.close();
                break;
            default:
                throw new IllegalArgumentException("未知的 uri");
        }
        db.close();

this.getContext().getContentResolver().notifyChange(Uri.parse("content://co
m.example.employee/insertemployee"),null);
        return uri;
```

```
    }

    @Override
    public int delete(@NonNull Uri uri, @Nullable String s, @Nullable String[]
strings) {
        int ret=-1;
        SQLiteDatabase db=employeeDatabaseHelper.getReadableDatabase();
        switch (matcher.match(uri)){
            case 1:
                ret = db.delete("employee",s,strings);
                break;
            case 2:
                ret = db.delete("employee",null,null);
                break;
            default:
                throw new IllegalArgumentException("未知的 uri");
        }
        db.close();

this.getContext().getContentResolver().notifyChange(Uri.parse("content://co
m.example.employee/employeeall"),null);
        return ret;
    }

    @Override
    public int update(@NonNull Uri uri, @Nullable ContentValues contentValues,
@Nullable String s, @Nullable String[] strings) {
        int ret=-1;
        SQLiteDatabase db=employeeDatabaseHelper.getReadableDatabase();
        switch (matcher.match(uri)){
            case 1:
                ret=db.update("employee",contentValues,s,strings);
                break;
            case 2:
                throw new IllegalAccessError("未知的 uri");
        }
        db.close();

this.getContext().getContentResolver().notifyChange(Uri.parse("content://co
m.example.employee/employeeall"),null);
```

```
        return ret;
    }
}
```

**EmployeeAdapter.java**

```
public class EmployeeAdapter extends SimpleAdapter {
    private ArrayList<Map<String,String>> mData;
    public ArrayList<Map<String,String>> getmData(){
        return mData;
    }
    public EmployeeAdapter(Context context, List<? extends Map<String, ?>>
data, int resource, String[] from, int[] to) {
        super(context, data, resource, from, to);
        this.mData=(ArrayList<Map<String,String>>)data;
    }
    public View getView(int position, View  convertView, ViewGroup parent){
        final int mPosition =position;
        return super.getView(position, convertView,parent);
    }
}
```

**EmployeeActivity.java**

```
/**
 * Created by 27682 on 2022/11/27.
 * 设计一个员工信息录入界面，当用户录入员工工号，姓名，身高等信息后支持数据的添加、
删除、更新和查询等基本操作。
 * 自定义一个ContentProvide，将这些数据暴露出去。
 */

public class EmployeeActivity extends AppCompatActivity {
    Button btnAdd,btnListAll,btnClearAll,btnDeleteAll,btnDelete,btnUpdate;
    EditText editId,editName,editAge,editHeight;
    ListView listView;
    String currentEmployeeId;

    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.ex9_employee_activity);
        btnDelete=(Button)findViewById(R.id.ex9btn_employee_delete);
        btnUpdate=(Button)findViewById(R.id.ex9btn_employee_update);
        btnAdd=(Button)findViewById(R.id.ex9btn_employee_add);
```

```
        btnClearAll=(Button)findViewById(R.id.ex9btn_employee_clearAll);
        btnDeleteAll=(Button)findViewById(R.id.ex9btn_employee_deleteAll);
        btnListAll=(Button)findViewById(R.id.ex9btn_employee_listAll);
        editAge=(EditText)findViewById(R.id.ex9edit_employee_age);
        editId=(EditText)findViewById(R.id.ex9edit_employee_id);
        editHeight=(EditText)findViewById(R.id.ex9edit_employee_height);
        editName=(EditText)findViewById(R.id.ex9edit_employee_name);
        listView=(ListView)findViewById(R.id.ex8_listView_employee);

        View view=
LayoutInflater.from(this).inflate(R.layout.employee_item_header,null);
        listView.addHeaderView(view);
        listView.setOnItemClickListener(new
AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
                if(position==0) {//=0 表示点击的是表头，不是内容，不用显示。否则表
头也会显示到上面的输入框中
                    editAge.setText("");
                    editId.setText("");
                    editHeight.setText("");
                    editName.setText("");
                    return;
                }
                TextView
employeeId=(TextView)view.findViewById(R.id.employeeId);
                TextView
employeeName=(TextView)view.findViewById(R.id.employeeName);
                TextView
employeeAge=(TextView)view.findViewById(R.id.employeeAge);
                TextView
employeeHeight=(TextView)view.findViewById(R.id.employeeHeight);
                editAge.setText(employeeAge.getText().toString());
                editId.setText(employeeId.getText().toString());
                editHeight.setText(employeeHeight.getText().toString());
                editName.setText(employeeName.getText().toString());
                currentEmployeeId=employeeId.getText().toString();//把当前
studentId 保存下来，当修改时当做原来的 studentId 使用
                Toast.makeText(EmployeeActivity.this,"编号输入框被锁定，点击更新
```

```
后可编辑",Toast.LENGTH_LONG).show();
                editId.setFocusable(false);//锁定编号输入框
                editId.setFocusableInTouchMode(false);

            }
        });
        btnClearAll.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                List<Map<String,String>> list=new ArrayList<Map<String,
String>>();
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
t});
                listView.setAdapter(adapter);
            }
        });
        btnDeleteAll.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                /*根据编号进行删除*/
//                String id=editId.getText().toString();
//                EmployeeDao.delete(EmployeeActivity.this,id);
                /*删除全部*/
                EmployeeDao.deleteAll(EmployeeActivity.this);
                /*重新查询全部*/
                List<Map<String,String>>
list=EmployeeDao.listAll(EmployeeActivity.this);
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
t});
                listView.setAdapter(adapter);
            }
        });
        btnListAll.setOnClickListener(new View.OnClickListener(){
```

```
            @Override
            public void onClick(View view) {
                List<Map<String,String>>
list=EmployeeDao.listAll(EmployeeActivity.this);
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
t});
                listView.setAdapter(adapter);
            }
        });
        btnAdd.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View view) {
                String id=editId.getText().toString();
                String name=editName.getText().toString();
                String age=editAge.getText().toString();
                String height=editHeight.getText().toString();
                try {

EmployeeDao.insert(EmployeeActivity.this,id,name,age,height);
                }catch (Exception e){
                    e.getMessage();
                    if(e.getMessage().equals("UNIQUE constraint failed:
employee.employeeid (code 1555)")){
                        System.out.println("主键值被重复插入");
                        Toast.makeText(EmployeeActivity.this,"主键值被重复插入
",Toast.LENGTH_LONG).show();
                    }
                    System.out.println(e.getMessage());
                }

                List<Map<String,String>>
list=EmployeeDao.listAll(EmployeeActivity.this);
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
t});
```

```
                listView.setAdapter(adapter);
            }
        });
        btnUpdate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String id=editId.getText().toString();
                String name=editName.getText().toString();
                String age=editAge.getText().toString();
                String height=editHeight.getText().toString();
                EmployeeDao.update(EmployeeActivity.this,id,name,age,height);
         /*重新查询并绑定*/
                List<Map<String,String>>
list=EmployeeDao.listAll(EmployeeActivity.this);
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
t});
                listView.setAdapter(adapter);
                editId.setFocusable(true);//解除限制，更新完毕后可以对编号进行修
改
                editId.setFocusableInTouchMode(true);

            }
        });
        btnDelete.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                 /*获取要删除的编号*/
                String id=editId.getText().toString();
        /*根据获取到的编号进行删除*/
                EmployeeDao.delete(EmployeeActivity.this,id);
        /*重新查询并绑定*/
                List<Map<String,String>>
list=EmployeeDao.listAll(EmployeeActivity.this);
                EmployeeAdapter adapter=new
EmployeeAdapter(EmployeeActivity.this,list,R.layout.employee_item_header,ne
w String[]{ "employeeid","employeename","employeeage","employeeheight"},new
int[]{R.id.employeeId,R.id.employeeName,R.id.employeeAge,R.id.employeeHeigh
```

```
t});
                listView.setAdapter(adapter);
                editId.setFocusable(true);//解除限制，更新完毕后可以对编号进行修改
                editId.setFocusableInTouchMode(true);
            }
        });
    }
}
```

（布局文件）**ex9_employee_activity.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="编号："/>
        <EditText
            android:id="@+id/ex9edit_employee_id"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="姓名："/>
        <EditText
            android:id="@+id/ex9edit_employee_name"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="年龄："/>
    <EditText
        android:id="@+id/ex9edit_employee_age"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="身高："/>
    <EditText
        android:id="@+id/ex9edit_employee_height"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    </LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button
        android:id="@+id/ex9btn_employee_add"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="添加数据"
        />
    <Button
        android:id="@+id/ex9btn_employee_listAll"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="全部显示"
        />
    <Button
        android:id="@+id/ex9btn_employee_clearAll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="清除显示"
        />
    <Button
        android:id="@+id/ex9btn_employee_deleteAll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="全部删除"
        />
</LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button
        android:id="@+id/ex9btn_employee_update"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:onClick="btnUpdate"
        android:text="更新"
        />
    <Button
        android:id="@+id/ex9btn_employee_delete"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:onClick="btnDelete"
        android:text="删除"
        />
</LinearLayout>
<ListView
    android:id="@+id/ex8_listView_employee"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"></ListView>
```

```
</LinearLayout>
```

**（listView 的布局文件）employee_item_header**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <!--  设控件的宽度为0，然后设置layout_weight，让他们按比例瓜分整个这一行的宽度,从而保证每一行的各列宽度都一样 -->
    <TextView
        android:id="@+id/employeeId"
        android:text="工号"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:textSize="20dp"
        android:textColor="#f0f"
        android:paddingLeft="10dp"
        android:layout_weight="1"/>
    <TextView
        android:id="@+id/employeeName"
        android:text="姓名"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:textSize="20dp"
        android:textColor="#f0f"
        android:paddingLeft="10dp"
        android:layout_weight="2"/>
    <TextView
        android:id="@+id/employeeAge"
        android:text="年龄"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:textSize="20dp"
        android:textColor="#f0f"
        android:paddingLeft="10dp"
        android:layout_weight="1"/>
    <TextView
        android:id="@+id/employeeHeight"
        android:text="身高"
```

```
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:textSize="20dp"
        android:textColor="#f0f"
        android:paddingLeft="10dp"
        android:layout_weight="1"/>
</LinearLayout>
```

**接收者**

Contentproviderdemo/ **AndroidManifest.xml**（接收）

```
<!--ContentProvider 的案例：调用另外一个应用程序（app2 中）自定义的 ContentProvider
-->
   <activity android:name=".StudentActivity">

   </activity>
<activity android:name=".EmployeeActivity_remote"
    android:label="实验九_姚圣杰">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

**第二部分：通过系统 contentProvider 远程操作手机联系人**

ContactActivity.java

```
/*
* 使用系统 ContentProvider，实验九的第二题，调用系统的 uri 查看手机通讯录
* */
public class ContactActivity extends AppCompatActivity {
    EditText txtContactId,txtContactName,txtPhoneNumber,txtEmail;
    //Button btnAdd,btnUpdate,btnDelete,btnQuery;
    ListView listViewContact;
    ContentResolver contentResolver;
    //Uri contactUri= Uri.parse("content://com.android.contacts/contacts");//
联系人信息 Uri
    Uri contactUri=ContactsContract.Contacts.CONTENT_URI;//联系人信息 Uri  推
荐使用这种方式
    //Uri
```

```
phoneUri=Uri.parse("content://com.android.contacts/data/phones");//联系人电话 Uri
    Uri phoneUri=ContactsContract.CommonDataKinds.Phone.CONTENT_URI;//联系人电话 Uri
    //Uri emailUri=Uri.parse("content://com.android.contacts/data/emails");//联系人 email Uri
    Uri emailUri=ContactsContract.CommonDataKinds.Email.CONTENT_URI;//联系人 email Uri
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_contact);
        listViewContact=(ListView)findViewById(R.id.listViewContact);
        txtContactId=(EditText)findViewById(R.id.txtContactId);
        txtContactName=(EditText)findViewById(R.id.txtContactName);
        txtPhoneNumber=(EditText)findViewById(R.id.txtPhoneNumber);
        txtEmail=(EditText)findViewById(R.id.txtEmail);
        contentResolver=getContentResolver();
         //下面开始给 listViewContact 添加 header。先把布局文件 contact_item_header 实例化为一个 view 对象
        View headerView=LayoutInflater.from(this).inflate(R.layout.contact_item_header,null);
        listViewContact.addHeaderView(headerView);
        listViewContact.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
                if(position==0) {//=0 表示点击的是表头，不是内容，不用显示。否则表头也会显示到上面的输入框中
                    txtContactId.setText("");
                    txtContactName.setText("");
                    txtPhoneNumber.setText("");
                    txtEmail.setText("");
                    return;
                }
                TextView contactId=(TextView)view.findViewById(R.id.contactId);
                TextView
```

```
contactName=(TextView)view.findViewById(R.id.contactName);
                TextView
phoneNumber=(TextView)view.findViewById(R.id.phoneNumber);
                TextView email=(TextView)view.findViewById(R.id.email);
                txtContactId.setText(contactId.getText().toString());
                txtContactName.setText(contactName.getText().toString());
                txtPhoneNumber.setText(phoneNumber.getText().toString());
                txtEmail.setText(email.getText().toString());
            }
        });
    }
    public void btnAddClick(View view){
        //先向ContactsContract.RawContacts.CONTENT_URI 插入一个空值，
        // 或者Uri uri =
Uri.parse("content://com.android.contacts/raw_contacts"); 就是raw_contacts
对应的uri
        // 目的是获得一个rawContactId（实际是raw_contact 表的_id 字段的值，是自增
的），
        // 后面要基于该rawContactId，插入其他字段值
        ContentValues values=new ContentValues();
        Uri
rawContactUri=contentResolver.insert(ContactsContract.RawContacts.CONTENT_U
RI,values);//先得到一个id，id 在联系人表中是自增的
        long rawContactId= ContentUris.parseId(rawContactUri);
        //开始准备插入联系人名字
        values.clear();
        //插入联系人的id
        values.put(ContactsContract.Data.RAW_CONTACT_ID,rawContactId);//联系
人的Id
        //插入内容类型：联系人姓名
        values.put(ContactsContract.Data.MIMETYPE,
ContactsContract.CommonDataKinds.StructuredName.CONTENT_ITEM_TYPE);//表示联
系人姓名
        //联系人名字

values.put(ContactsContract.CommonDataKinds.StructuredName.GIVEN_NAME,txtCo
ntactName.getText().toString().trim());
        //插入联系人名字
        contentResolver.insert(ContactsContract.Data.CONTENT_URI,values);
        // 下面开始准备插入联系人电话号码
```

```
        values.clear();
        values.put(ContactsContract.Data.RAW_CONTACT_ID,rawContactId);
        //插入内容类型：联系人电话号码
        values.put(ContactsContract.Data.MIMETYPE,
ContactsContract.CommonDataKinds.Phone.CONTENT_ITEM_TYPE);//表示电话号码

values.put(ContactsContract.CommonDataKinds.Phone.NUMBER,txtPhoneNumber.get
Text().toString().trim());
        //电话号码的类型
        values.put(ContactsContract.CommonDataKinds.Phone.TYPE,
ContactsContract.CommonDataKinds.Phone.TYPE_MOBILE);//移动电话
        contentResolver.insert(ContactsContract.Data.CONTENT_URI,values);
        //下面开始准备插入 email
        values.clear();
        values.put(ContactsContract.Data.RAW_CONTACT_ID,rawContactId);
        //插入内容的类型：email
        values.put(ContactsContract.Contacts.Data.MIMETYPE,
ContactsContract.CommonDataKinds.Email.CONTENT_ITEM_TYPE);//表示 Email

values.put(ContactsContract.CommonDataKinds.Email.DATA,txtEmail.getText().t
oString().trim());
        // 设置该电子邮件的类型
        values.put(ContactsContract.CommonDataKinds.Email.TYPE,
ContactsContract.CommonDataKinds.Email.TYPE_WORK);
        contentResolver.insert(ContactsContract.Data.CONTENT_URI,values);
        Toast.makeText(this, "联系人数据添加成功", Toast.LENGTH_SHORT).show();
    }
    public void btnUpdateClick(View view){
       /* (1)不需要更新raw_contacts，只需要更新data 表；
        (2)uri=content://com.android.contacts/data 表示对data 表进行操作；
        */
        Uri uri=Uri.parse("content://com.android.contacts/data");
        ContentValues values=new ContentValues();
        values.put("data1",txtContactName.getText().toString().trim());
        contentResolver.update(uri,values,"mimetype=? and
raw_contact_id=?",new
String[]{"vnd.android.cursor.item/name",txtContactId.getText().toString()})
;
        values.clear();
```

```
values.put(ContactsContract.CommonDataKinds.Phone.NUMBER,txtPhoneNumber.get
Text().toString().trim());
        values.put(ContactsContract.Data.MIMETYPE,
ContactsContract.CommonDataKinds.Phone.CONTENT_ITEM_TYPE);
        values.put(ContactsContract.CommonDataKinds.Phone.TYPE,
ContactsContract.CommonDataKinds.Phone.TYPE_MOBILE);//移动电话
        contentResolver.update(uri,values,"mimetype=? and
raw_contact_id=?",new
String[]{"vnd.android.cursor.item/phone_v2",txtContactId.getText().toString
()});

        values.clear();
        values.put("data1",txtEmail.getText().toString().trim());
        //或者
values.put(ContactsContract.CommonDataKinds.Email.DATA,txtEmail.getText().t
oString().trim());
        values.put(ContactsContract.Contacts.Data.MIMETYPE,
ContactsContract.CommonDataKinds.Email.CONTENT_ITEM_TYPE);//MIMETYPE=1,表示
Email
        values.put(ContactsContract.CommonDataKinds.Email.TYPE,
ContactsContract.CommonDataKinds.Email.TYPE_WORK);
        contentResolver.update(uri,values,"mimetype=? and
raw_contact_id=?",new
String[]{"vnd.android.cursor.item/email_v2",txtEmail.getText().toString()})
;

    }

    public void btnDeleteClick(View view){
        String contactId=txtContactId.getText().toString().trim();
        //删除数据，把raw_contact 表中id 等于要删除的id 值的记录删除即可。
        //因而uri 使用ContactsContract.RawContacts.CONTENT_URI，跟增加联系人是
一样的uri

if(contentResolver.delete(ContactsContract.RawContacts.CONTENT_URI,Contacts
Contract.RawContacts._ID+"="+contactId,null)>=1){

            Toast.makeText(ContactActivity.this,"delete "+contactId+"  成功
",Toast.LENGTH_LONG).show();
        }
```

```
    }
    public void btnQueryClick(View view){
        Cursor cursor;
        List<Map<String,String>> listContact=new
ArrayList<Map<String,String>>();
        String contactId;
        StringBuffer sb=new StringBuffer();//用来临时保存联系人的多个电话和多个
email
        String name=txtContactName.getText().toString().trim();
        cursor=contentResolver.query(contactUri,new
String[]{ContactsContract.Contacts._ID,ContactsContract.Contacts.DISPLAY_NA
ME},ContactsContract.Contacts.DISPLAY_NAME+ " like '%"+name+"%'",null,null);
        while(cursor.moveToNext()){//遍历所有联系人表，有 2 个字段，一个 id，一个
displayName，分别用 ContactsContract.Contacts._ID 和
ContactsContract.Contacts.DISPLAY_NAME 代表
            //1 得到联系人表的 id 和 displayName

contactId=cursor.getString(cursor.getColumnIndex(ContactsContract.Contacts.
_ID));
            String
contactName=cursor.getString(cursor.getColumnIndex(ContactsContract.Contact
s.DISPLAY_NAME));
            //2 查询该 id 对应的电话表中的电话号码，电话号码字段是
ContactsContract.CommonDataKinds.Phone.NUMBER 所代表的值表示的
            Cursor phones=contentResolver.query(phoneUri,new
String[]{ContactsContract.CommonDataKinds.Phone.NUMBER},ContactsContract.Co
mmonDataKinds.Phone.CONTACT_ID+"="+contactId,null,null);
            while (phones.moveToNext()){

sb.append(phones.getString(phones.getColumnIndex(ContactsContract.CommonDat
aKinds.Phone.NUMBER)));
                sb.append("  ");//每个号码之间用两个空格隔开
            }
            phones.close();
            String phoneNumber=sb.toString().trim();//去掉最后的一个电话号码之
后加上的 2 个空格
            //3 查询该 id 对应 email 表中的 email，email 字段是
ContactsContract.CommonDataKinds.Email.DATA 所代表的值表示的
```

```
            sb.delete(0,sb.length()-1);
            Cursor emails=contentResolver.query(emailUri,new
String[]{ContactsContract.CommonDataKinds.Email.DATA},ContactsContract.Comm
onDataKinds.Email.CONTACT_ID+"="+contactId,null,null);
            while(emails.moveToNext()){

sb.append(emails.getString(emails.getColumnIndex(ContactsContract.CommonDat
aKinds.Email.DATA)));
                sb.append("  ");
            }
            emails.close();
            String email=sb.toString().trim();//去掉最后的一个email 之后加上的2
个空格
            sb.delete(0,sb.length()-1);
            //下面开始生成Map 对象
            Map<String,String> map=new HashMap<String,String>();
            map.put("contactId",contactId);
            map.put("contactName",contactName);
            map.put("phoneNumber",phoneNumber);
            map.put("email",email);
            listContact.add(map);
        }
        cursor.close();

        //联系人遍历完毕，也生成了联系人列表listContact，每一行联系人包括4 项：联
系人id、联系人名称、电话号码、email
        //开始生成SimpleAdapter 对象
        SimpleAdapter adapter=new
SimpleAdapter(ContactActivity.this,listContact,R.layout.contact_item,new
String[]{"contactId","contactName","phoneNumber","email"},new
int[]{R.id.contactId,R.id.contactName,R.id.phoneNumber,R.id.email});
        listViewContact.setAdapter(adapter);
        System.out.println("查询联系列表结束，并显示到了 listViewContact 上");
    }
    static void setEditTextReadOnly(EditText editText){
        editText.setCursorVisible(false);//设置光标不可见
        editText.setFocusable(false);//设置得不到输入焦点
        editText.setFocusableInTouchMode(false);//设置触摸时也得不到焦点
    }
    static void setEditTextEditable(EditText editText){
```

```
        editText.setCursorVisible(true);//设置光标可见
        editText.setFocusable(true);//设置可以得到输入焦点
        editText.setFocusableInTouchMode(true);//设置触摸时也可以得到焦点
    }
}
```

（布局文件）**ex9_employee_activity.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="编号："/>
        <EditText
            android:id="@+id/ex9edit_employee_id"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />

        </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="姓名："/>
        <EditText
            android:id="@+id/ex9edit_employee_name"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="年龄："/>
        <EditText
            android:id="@+id/ex9edit_employee_age"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="身高："/>
        <EditText
            android:id="@+id/ex9edit_employee_height"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button
            android:id="@+id/ex9btn_employee_delete"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="btnDeleteClick"
            android:text="ID 删除"
            />
        <Button
            android:id="@+id/ex9btn_employee_listAll"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="btnListAllClick"
```

```
            android:text="显示全部"
            />
        <Button
            android:id="@+id/ex9btn_employee_query"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="btnQueryClick"
            android:text="ID 查询"
            />
        <Button
            android:id="@+id/ex9btn_employee_update"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:onClick="btnUpdateClick"
            android:text="ID 更新"
            />

    </LinearLayout>
    <ListView
        android:id="@+id/ex8_listView_employee"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"></ListView>


</LinearLayout>
```

## 五、实验总结

实验内容分成三个部分，第一和第二部分别分围绕着将本地的操作使用自定义 ContentProvide 的暴露和远程操作。第三部分为

在进行**第一部分**对本地操作的添加数据，全部显示，清除显示，全部删除等功能的实现过程中，对 SQLite 的使用有了进一步的掌握。实现思路大致为：新建 EmployeeDatabaseHelper，建表"create table employee(employeename text primary key,employeeage,employeeheight)"；新建 EmployeeContentProvider，实现 CRUD 方法。

# 山东建筑大学 计算机学院 实验报告

班级：软件 201 姓名：姚圣杰 学 号：202010101067 实验成绩：

课程：移动设备开发 A 同组者： 实验日期：2022/11/29

---

在自定义 ContentProvider 时，对 extends ContentProvider 时需要重新的方法进行了编写的练习，对使用入参的方式操作 SQLite 数据库的方式有了进一步的练习和掌握。使用 matcher.addURI()进行了 Uri 的注册。

在**第二部分**，使用第一部分所暴露出的 ContentProvider 进行远程的数据库操作。使用 contentResolver.registerContentObserver，Uri.parse，contentResolver.query，contentResolver.query 等方法与已经注册的 Uri 进行匹配。并根据进行远程操作的 activity 的 editText 提供的入参进行操作。

**第三部分**，

通过在 manifests 中进行权限的注册和在 acitivity 中调用系统 Uri 对联系人进行增删改查。

打开访问“联系人”权限的步骤：设置-应用与通知-应用权限中的自定义 app 的通讯录权限需要开放，才可以调用通讯录的 ContentProvider。